AI Interview Coach: Optimizing Software Engineering Interview Feedback through RLAIF and DPO

Chun-Han Chien Yun-Chen Huang Masaki Kitagawa Deng-Chyi Yang
Oregon State University
{chiench, huangyun, kitagawm, yangden}@oregonstate.edu

Abstract

Preparing for software engineering interviews is a critical but resource-intensive process for computer science students. While large language models (LLMs) offer a scalable solution for automated coaching, their feedback often lacks the specificity and actionability required for high-stakes technical interviews. In this project, we build an AI Interview Coach based on Qwen2.5-3B-Instruct and attempt to improve it through Reinforcement Learning from AI Feedback (RLAIF) and Direct Preference Optimization (DPO), using a stronger judge model (Qwen3.5-9B) to generate preference data over a five-dimension coaching rubric. On a fixed 200-example test set, the SFT+DPO model attains a marginally higher mean judge score than the baseline (17.03 vs. 16.86 on a 1–20 scale), but the difference is not statistically significant (paired t-test $p = 0.42$; Wilcoxon $p = 0.70$; Cohen's $d = 0.057$). A per-item failure analysis shows that genuine gains—notably the removal of baseline hallucinations—are offset by newly introduced technical errors, and that a strong ceiling effect (65% of baseline outputs already score $\geq 18/20$) together with a coarse, homogeneous evaluation set limits the measurable benefit. We discuss the methodological implications for evaluating preference optimization with LLM judges.

1. Introduction

The technical interview is a gatekeeping milestone in the software engineering industry. Success requires not only strong algorithmic knowledge but also the ability to communicate complex technical concepts clearly and efficiently. However, many students, particularly at the graduate level, lack access to personalized, high-quality coaching that can identify subtle gaps in their technical explanations or communication styles.

Traditional automated systems often provide generic feedback that fails to address the specific nuances of a student's answer. Large language models (LLMs) show promise in this domain, yet base models often struggle to maintain a consistent coaching persona that is both technically rigorous and pedagogically supportive.

This work aims to bridge this gap by fine-tuning a lightweight, efficient LLM (Qwen2.5-3B) to act as a specialized interview coach. By leveraging a more powerful judge model (Qwen3.5-9B) to provide feedback on feedback (RLAIF), we create a robust dataset of preferences that prioritize technical precision and actionable advice. We use Direct Preference Optimization (DPO) with QLoRA to refine the model's behavior, ensuring it provides feedback that is specifically tailored to the requirements of software engineering interviews.

2. Related Work

RLHF and RLAIF. Reinforcement Learning from Human Feedback (RLHF) has been the standard for aligning LLMs with human values [1]. However, gathering human preferences is slow and expensive. Reinforcement Learning from AI Feedback (RLAIF) has emerged as a viable alternative, where a stronger "teacher" model provides preferences to train a "student" model, often achieving performance comparable to human-labeled data [2].

Direct Preference Optimization. DPO simplifies the RLHF pipeline by optimizing the policy model directly from preference pairs without the need for a separate reward model [3]. This makes it particularly suitable for constrained compute environments where training a complex reward model is impractical.

AI in Education. Recent work has explored LLMs for automated grading and feedback in programming and STEM education. LLM-based feedback can draw on intelligent tutoring systems and learning sciences to improve pedagogical quality [4]. Since our pipeline relies on an LLM judge, we also consider recent findings on aligning LLM-assisted evaluation with human preferences [5]. However, most existing tools focus on code correctness rather than the conversational and descriptive technical communication required in interviews.

3. Methodology

Our pipeline consists of four phases: (i) dataset construction and student-answer simulation, (ii) preference-candidate generation with bias control, (iii) preference labeling via an LLM judge (RLAIF), and (iv) model optimization with SFT and DPO. Throughout, the policy model that is fine-tuned is Qwen2.5-3B-Instruct, and a stronger Qwen3.5-9B model serves as the judge.

3.1. Dataset and Student Simulation

We build the dataset from two public software engineering interview question sources: the *Software Engineering Interview Questions* dataset [6] and the *ai_interview_questions* set. After preprocessing we obtain 4,653 training examples and a held-out test set of 200 examples. Because preference candidate generation, two-pass AI judging, SFT, and DPO training are all computationally expensive under our Colab/GPU-time constraints, we did not use the full training pool for the preference optimization stage. Instead, we randomly sampled 2,000 examples from the 4,653 training examples for preference candidate generation and DPO training. These 2,000 examples were re-indexed from `train_0001` to `train_2000`. The held-out 200-example test set was kept unchanged throughout the project and was reused for both baseline and final evaluation. Each example pairs an interview question with a reference answer, category and difficulty metadata, and a simulated student answer.

To probe the coach under realistic conditions, we simulate answers from a fixed persona—a first-year CS Master's student with strong fundamentals but limited industry experience, prone to omitting edge cases and using imprecise terminology. Using this persona, we prompt the base model to produce five student-answer types spanning the common failure modes:

- **Correct but incomplete:** missing edge cases or optimizations.
- **Partially correct:** a sound core with notable gaps or errors.
- **Incorrect:** a fundamentally wrong approach.
- **Too vague:** lacking technical depth or specific terminology.
- **Verbose but unfocused:** burying the answer in irrelevant detail.

Crucially, the simulated student answers and the test questions are generated *once* and then frozen: the identical inputs are reused for baseline evaluation, preference generation, and final evaluation. This design isolates the coaching model as the only variable, so any score difference is attributable to fine-tuning rather than to changes in the inputs. We note, however, that in the released data used for the experiments in this paper every train and test item carries the single label `partially_correct`, and category/difficulty metadata is `Unknown` for most items; the evaluation therefore exercises only this subset of the intended answer types (see Sec. 5).

3.2. Preference Candidate Generation

For each training example, the base model generates two coaching feedback candidates (`feedback_a`, `feedback_b`) for the same fixed student answer. To prevent the preference data from encoding superficial cues, we draw each candidate from a randomized but reproducible feedback style, sampled from a pool of distinct coaching styles—*technical precision focus*, *interview communication focus*, *balanced coaching* (strengths/weaknesses/advice), and *supportive yet strict*—generated at varied temperatures. The A/B slot is not tied to a fixed quality level, and we log the style, temperature, and candidate model for each side. This reduces the risk that the downstream model learns spurious patterns such as preferring the second candidate, longer responses, or a particular style.

3.3. Preference Labeling via RLAIF

We employ Qwen3.5-9B as the judge. It scores feedback on five dimensions—Technical Correctness, Specificity, Helpfulness, Actionability, and Interview Coaching Quality—each on a 1–20 scale, with the top band (18–20) reserved for "exceptional" feedback containing concrete code or master-class guidance. In our initial experiments, direct A/B preference selection showed a risk of position bias: the judge could favor one side partly because of whether it appeared as response A or response B, rather than only because of coaching quality. To reduce this bias, we changed the preference selection protocol to a two-pass swapped-order average-score method. For each feedback pair, the judge first scored feedback_a and feedback_b in the original order on the same 1–20 rubric. We then ran the same comparison again with the order swapped. For each original candidate, we averaged its scores across the two passes. The candidate with the higher average score was selected as the chosen response, and the lower-scoring candidate was used as the rejected response. This final two-pass swapped-order average-score method was used to construct the 2,000-example preference pair dataset for DPO.

To ensure consistent and structural output, we implemented several prompt engineering refinements. We found that base judge models often outputted lengthy "thinking" monologues or conversational fillers that interfered with automated JSON parsing. We addressed this using *assistant pre-filling*: by appending the starting character of a JSON object ({) to the model's prompt, we successfully

suppressed the monologue and forced the model to immediately begin generating the structured score data.

The judge scores feedback on five dimensions: Technical Correctness, Specificity, Helpfulness, Actionability, and Interview Coaching Quality. Initially, we experimented with a 1–5 Likert scale, but found it lacked the resolution to distinguish subtle improvements in coaching quality. We therefore expanded the evaluation to a 1–20 scale and implemented a rigorous calibration rubric to combat leniency bias. Under this protocol, scores of 10–12 represent standard/good feedback that is accurate but generic; 15–17 represent clearly strong feedback with precise gap identification; and the top band (18–20) is strictly reserved for "exceptional" master-class coaching containing tailored code snippets or flawless guidance. Crucially, the judge is instructed that any identified flaw or opportunity for elaboration must cap the score at 14, ensuring a high bar for excellence.

For preference selection, the judge compares candidates using a two-pass swapped-order protocol to mitigate *position bias*—the common tendency of LLMs to systematically favor the first or second response regardless of actual quality. If left unaddressed, this bias would introduce significant noise into the RLAIF pipeline, causing the model to optimize for arbitrary structural artifacts rather than genuine coaching merit. In the first pass, `feedback_a` is presented in the first slot and `feedback_b` in the second; in the second pass, the display order is inverted. The judge assigns independent 1–20 scores for each slot in both passes. We then calculate the final quality score for each candidate by averaging its score across both presentations (e.g., averaging the score of `feedback_a` when it was in the first slot with its score when it was in the second). This final two-pass swapped-order average-score method ensures that the *chosen* and *rejected* labels used for DPO reflect consistent, position-independent quality, and was used to construct the 2,000-example preference pair dataset for DPO.

3.4. Model Optimization

We fine-tune with QLoRA (Quantized Low-Rank Adaptation) on a Google Colab A100 instance, training lightweight adapter weights rather than the full model to fit our compute budget. Optimization proceeds in two stages:

1. **SFT (Supervised Fine-Tuning):** we first build a supervised dataset that maps the (question, reference answer, student answer) input to the judge-preferred *chosen* feedback, and train on it to establish the general coaching format and persona.
2. **DPO (Direct Preference Optimization):** starting from the SFT model, we train on the RLAIF-labeled *chosen/rejected* pairs to directly optimize for the coaching rubric without a separate reward model.

Dimension	Baseline	SFT+DPO	Δ
Technical Correctness	16.79	16.92	$+0.14$
Specificity	16.88	17.06	$+0.19$
Helpfulness	16.88	17.05	$+0.18$
Actionability	16.89	17.05	$+0.17$
Interview Coaching Quality	16.88	17.06	$+0.19$
Overall Score	**16.86**	**17.03**	$\mathbf{+0.17}$

Table 1. Mean judge scores (1–20) on the 200-example test set. The trained model is consistently but marginally higher across all dimensions.

4. Experimental Results

Evaluation protocol. We evaluate both models on a fixed held-out test set of 200 examples. The test questions and the simulated student answers are identical for the baseline and the trained model, so any score difference is attributable only to the coaching model. For each example, the judge model (Qwen3.5-9B) scores the feedback on five dimensions (Technical Correctness, Specificity, Helpfulness, Actionability, Interview Coaching Quality), each on a 1–20 scale; the overall score is their mean. We report per-dimension means and, because the same 200 items are scored by both models, we treat the two score vectors as *paired* samples and test the difference with a paired t-test, the Wilcoxon signed-rank test (robust to the non-normal, ceiling-bound score distribution), and an exact binomial sign test on the non-tie items. We also report Cohen's d for paired samples as an effect size.

Aggregate results. As shown in Tab. 1, the SFT+DPO model attains a marginally higher mean overall score than the baseline (17.03 vs. 16.86, $\Delta = +0.17$ on the 1–20 scale). The improvement is consistent in direction across all five dimensions, but small in magnitude (per-dimension Δ between $+0.14$ and $+0.19$).

Statistical significance. The improvement is *not* statistically significant. For the overall score, the paired t-test gives $t = 0.81$, $p = 0.42$, and the Wilcoxon signed-rank test gives $p = 0.70$; the effect size is negligible (Cohen's $d = 0.057$). No individual dimension reaches significance (all $p > 0.38$; see Tab. 2). On an item-by-item basis the trained model wins on 48 examples, loses on 40, and ties on 112; the sign test over the 88 non-tie items is also non-significant ($p = 0.46$). We therefore cannot reject the null hypothesis that SFT+DPO leaves feedback quality unchanged under this judge and rubric.

5. Discussion

The absence of a significant difference is itself an informative result. We identify three contributing factors and

Dimension	Baseline (SD)	SFT+DPO (SD)	Δ	Paired t (p)	Wilcoxon p	Cohen's d
Technical Correctness	16.79 (2.40)	16.92 (1.92)	$+0.14$	0.63 (0.53)	0.73	0.044
Specificity	16.88 (2.37)	17.06 (1.82)	$+0.19$	0.88 (0.38)	0.65	0.062
Helpfulness	16.88 (2.37)	17.05 (1.85)	$+0.18$	0.83 (0.41)	0.69	0.058
Actionability	16.89 (2.38)	17.05 (1.84)	$+0.17$	0.78 (0.44)	0.81	0.055
Interview Coaching Quality	16.88 (2.37)	17.06 (1.82)	$+0.19$	0.88 (0.38)	0.69	0.062
Overall Score	**16.86 (2.37)**	**17.03 (1.85)**	$\mathbf{+0.17}$	$\mathbf{0.81}$ $\mathbf{(0.42)}$	$\mathbf{0.70}$	$\mathbf{0.057}$

Table 2. Paired statistical comparison of baseline vs. SFT+DPO over the 200 test examples. No dimension reaches significance at $\alpha = 0.05$, and all effect sizes are negligible ($d < 0.07$). Win/loss/tie on the overall score is 48/40/112; the sign test over non-tie items gives $p = 0.46$.

support them with a per-item failure analysis.

Ceiling effect. The judge already rates the baseline very highly, leaving little headroom for improvement. 65% of baseline outputs score $\geq 18/20$, and on 191 of 200 items *both* models score ≥ 15. When the starting point is already near the top of the scale, even a genuinely better model cannot move the mean by much.

Coarse judge resolution. The judge's scores are effectively discrete, clustering on a handful of values (predominantly 15 and 18), and the five rubric dimensions almost always move together (their per-item scores are nearly identical). This low resolution injects noise and masks the fine-grained differences that distinguish good coaching feedback from excellent coaching feedback.

Offsetting gains and regressions. The flat average hides real movement in both directions: 48 items improve, 40 regress, and these roughly cancel. The trained model's largest wins are recoveries of *catastrophic* baseline failures—three items jump from 1 to 18 (`test_0001`, `test_0073`, `test_0119`) by removing a baseline hallucination that claimed the student had missed a concept actually present in the answer. However, the trained model also introduces *new* technical errors that the baseline did not make: in the most severe regression, `test_0194` drops from 18 to 1 because the feedback falsely asserts that Java's `@Override` annotation throws a runtime `java.lang.Error`. Categorizing the judge's stated reason on the 40 regressions, 4 are clear technical inaccuracies and the remainder are smaller, mostly within the judge's own scoring noise (e.g. slightly less complete or less specific feedback).

Test-set diversity limitation. A confound limits what this experiment can demonstrate. Although our methodology defines five student-answer types (correct-but-incomplete, partially correct, incorrect, too vague, verbose but unfocused), in the released data all 200 test items (and all training items) carry the label `partially_correct`, and the question category/difficulty metadata is missing (`Unknown`) for 189 of the 200 items. The evaluation therefore probes only a single, narrow slice of the intended input distribution. A homogeneous test set both reduces the variety of coaching behaviors exercised and likely contributes to the ceiling effect, since "partially correct" answers tend to elicit similarly-structured feedback that the judge scores uniformly. Rebuilding the test set to span the full range of answer types and difficulties is the most important step toward a fairer and more sensitive comparison.

Implications. Two methodological lessons follow. First, an LLM judge that saturates near the top of its scale is a poor instrument for measuring incremental alignment gains; a more discriminative protocol—a finer or forced-spread rubric, or direct pairwise preference (win rate) judging rather than absolute scoring—would have more statistical power. Second, preference optimization that is not explicitly grounded against a reference answer can introduce fluent but false technical claims; pairing DPO with a factuality-checking signal is a promising direction. We discuss a small-scale human evaluation to validate the judge in future work.

6. Conclusion

We built an AI Interview Coach by fine-tuning Qwen2.5-3B with SFT and DPO on RLAIF-labeled preference data, and evaluated it against the baseline with an LLM judge on a fixed 200-example test set. The trained model scored marginally higher on every rubric dimension, but no difference was statistically significant, and the effect size was negligible. Our failure analysis explains why: the judge saturates near the top of its scale (a ceiling effect), its scores are low-resolution, the evaluation set is homogeneous (all test answers share a single answer-type label), and real improvements are cancelled by occasional new technical errors introduced during preference optimization. Rather than a clear win, our main contributions are negative but instructive: (i) LLM judges that saturate are weak instruments for detecting incremental alignment gains, motivating pairwise/win-rate evaluation and finer rubrics; (ii) reference-free DPO can introduce fluent but false technical claims, motivating a factuality-grounded preference signal. Future work includes rebuilding a diverse test set across all

student-answer types and difficulties, a small-scale human evaluation to validate the judge, and grounding the coach against reference answers.

References

[1] Long Ouyang et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022. 1

[2] Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022. 1

[3] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023. 1

[4] John Stamper, Ruiwei Xiao, and Xinying Hou. Enhancing LLM-based feedback: Insights from intelligent tutoring systems and the learning sciences. In *International Conference on Artificial Intelligence in Education*. Springer, 2024. 1

[5] Shreya Shankar, J.D. Zamfirescu-Pereira, Björn Hartmann, Aditya G. Parameswaran, and Ian Arawjo. Who validates the validators? aligning LLM-assisted evaluation of LLM outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, 2024. 1

[6] Syed Muhammad Haris. Software engineering interview questions dataset. https://www.kaggle.com/datasets/syedmharis/software-engineering-interview-questions-dataset, 2024. Kaggle. 2